

Lab 1: Introduction to Frontend Development & Grid Layouts

Course: CST150: Web Development & Databases

Objective

By the end of this lab, students will:

- Understand the basics of frontend development.
- Learn the structure of an HTML page.
- Explore CSS for styling.
- Implement a simple web page using CSS Grid for layout.

Don't just follow the workshop verbatim, tweak the changes and see the difference. Share your findings with the group.

Pre-requisites

- Basic knowledge of HTML and CSS.
-

Lab Tasks

Task 1: Setting Up Your Project

1. Open a text editor and create a new folder named `CST150_lab1`.
2. Inside this folder, create the following files:
 - `index.html`
 - `styles.css`
3. Open `index.html` and add the basic structure:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Frontend Grid Layout</title>
  <link rel="stylesheet" href="styles.css">
</head>
```

```
<body>
  <header>Header</header>
  <nav>Navigation</nav>
  <main>Main Content</main>
  <aside>Sidebar</aside>
  <footer>Footer</footer>
</body>
</html>
```

Task 2: Applying Basic CSS

1. Open `styles.css` and add basic styling:

```
body {
  font-family: Arial, sans-serif;
  margin: 0;
  padding: 0;
  display: grid;
  grid-template-areas:
    "header header"
    "nav main"
    "nav aside"
    "footer footer";
  grid-template-columns: 1fr 3fr;
  grid-template-rows: auto 1fr 1fr auto;
  height: 100vh;
}

header { grid-area: header; background: #333; color: white;
padding: 20px; text-align: center; }
nav { grid-area: nav; background: #444; color: white; padding:
20px; }
main { grid-area: main; background: #f4f4f4; padding: 20px; }
aside { grid-area: aside; background: #ddd; padding: 20px; }
footer { grid-area: footer; background: #333; color: white;
padding: 20px; text-align: center; }
```

Task 3: Testing Your Layout

1. Open `index.html` in a browser.
 2. Observe how the grid layout arranges different sections.
 3. Modify the grid structure by changing `grid-template-areas` and `grid-template-columns`.
-

Discussion Questions

1. How does CSS Grid help in structuring layouts?
 2. What are the benefits of using `grid-template-areas`?
 3. How can you make this layout responsive?
-

Bonus Challenge

- Modify the layout so that on smaller screens, the sidebar moves below the main content.
 - Use media queries to make the navigation a horizontal bar on larger screens and a vertical menu on smaller screens.
-

Conclusion

In this lab, students have learned how to set up a simple frontend project, create a structured HTML page, and use CSS Grid to design a responsive layout. Future lessons will dive deeper into styling, interactivity, and database integration.

End of Lab 1